FAST National University of Computer and Emerging Sciences
Karachi Campus

FINAL PROJECT REPORT

Lost and Found Management System

A menu-driven C console application for tracking lost and found items in an institutional setting, with file-based persistence and an automated matching feature.

#	Name	Student ID
1	Danish Raheem	25K-6514
2	Hiten Kalani	25K-6549

Course: Programming Fundamentals (CS-1002)
Section: BCE-2A / 2B
Semester: Spring 2026 — Semester 2
Submission Date: April 28, 2026

Abstract

This report presents the design and implementation of a Lost and Found Management System developed as a C console application. The system replaces the conventional paper-register approach to lost-and-found tracking in institutions with a structured, menu-driven program backed by plain-text file storage. Users can register lost items, record found items, search the database, view automatically suggested matches between lost and found records, and update item status to *returned*. The project applies the core Programming Fundamentals constructs of **structs, arrays, file I/O, loops, conditionals, and functions** to a practical, real-world problem.

Acknowledgment

We thank our Programming Fundamentals course instructor for guidance throughout the semester, and the lab demonstrators whose feedback helped shape the final design. We also acknowledge the academic environment of FAST NUCES Karachi, which made this project possible.

Table of Contents

1. Introduction

1.1 Background

Lost and found management is a routine but often overlooked administrative task in schools, universities, and offices. Misplaced items such as ID cards, keys, electronics, and personal belongings need to be tracked, matched with their owners, and returned in a timely manner. A simple, structured digital solution improves recovery rates and saves time for both staff and students.

1.2 Problem Statement

Most institutions still rely on paper registers, notice boards, or informal verbal communication to manage lost-and-found records. These methods suffer from several practical issues:

- Records are scattered, easily damaged, or lost.
- Searching for a specific item is slow and inefficient.
- Item status (lost / found / returned) is not consistently tracked.
- There is no organised process for matching lost items with found ones, leading to low return rates.

1.3 Objectives

- Build a menu-driven C console application for managing lost and found records.
- Use **structs** to model items and **file storage** for persistence between runs.
- Provide search functionality across multiple fields (name, category, date, location).
- Implement a simple matching algorithm to suggest likely lost-found pairs.
- Allow status updates and maintain an action log.
- Demonstrate core Programming Fundamentals concepts in a practical setting.

2. Scope of the Project

2.1 Inclusions

• A C console application with a menu-driven interface.

• Structs to model lost and found items with the fields: ID, name, category, description, date, location, contact, and status.

• File-based persistence using plain-text files (`lost_items.txt` and `found_items.txt`).

• Add, view, search, match, and update operations.

• A simple matching algorithm based on category, name, and date proximity.

• An action log file (`log.txt`) recording status changes.

• Summary statistics (counts of lost / found / returned).

2.2 Exclusions

• No graphical user interface; the system is console-only.

• No network or cloud connectivity.

• No multi-user authentication or role-based access.

• No image upload or attachment support.

• No integration with email or SMS notifications.

3. System Overview

3.1 Project Description

The Lost and Found Management System is a single-user C console application. On launch, the program loads existing records from text files into in-memory arrays of structs. The user is then presented with a numbered menu offering options to add new items, list existing records, search by various criteria, view suggested matches, and mark items as returned. Any change is written back to the corresponding file before the program exits, ensuring that data persists between sessions.

3.2 Application Flow

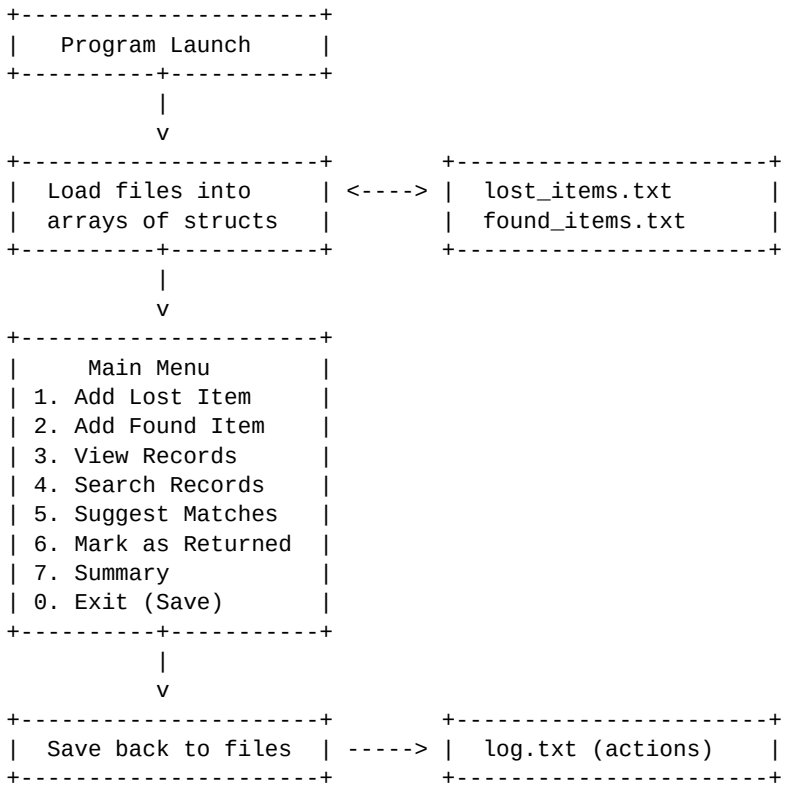

Figure 1. Application flow of the Lost and Found Management System.

4. Implementation Details

4.1 Data Structure

Each record in the system is represented by an **Item** struct. The same struct definition is used for both lost and found items; they are kept in separate arrays and separate files so that the matching logic can operate cleanly between the two sets.

```
typedef struct {
    int  id;
    char name[50];
    char category[30];
    char description[100];
    char date[15];
    char location[50];
    char contact[40];
    char status[15];   // "lost", "found", "returned"
} Item;
```

4.2 File Format

Records are written to plain-text files, one record per line, with fields separated by a pipe (`|`) character. This format is human-readable, easy to parse with `fscanf` / `fgets`, and simple to inspect during testing.

```
1|Black Wallet|Accessory|Leather, brown stitching|2026-04-22|Library|03001234567|lost
2|Casio Calculator|Electronics|Black, FX-991|2026-04-23|Lab-3|03219876543|found
```

4.3 Key Functions

- `loadItems()` — reads a text file into an array of Item structs.
- `saveItems()` — writes the array back to file before exit.
- `addItem()` — prompts the user for fields and appends a new record.
- `viewItems()` — prints all records in a formatted table.
- `searchItems()` — filters records by name, category, date, or location.
- `suggestMatches()` — compares lost and found arrays and prints likely pairs based on category and name similarity.
- `markReturned()` — updates the status of an item by ID and writes an entry to `log.txt`.

5. Methodology

5.1 Approach

An incremental, function-by-function development approach was followed. The data structure was finalised first, then file I/O was tested with small dummy datasets, and the menu-driven interface was layered on top. Search and matching features were added last, once the base data flow was stable.

5.2 Project Phases

Phase 1 — Design. Defined the Item struct, file format, and menu layout.

Phase 2 — Core Operations. Implemented add, view, and the load / save file routines.

Phase 3 — Search and Match. Added search filters across multiple fields and the matching algorithm.

Phase 4 — Status Tracking. Implemented status updates and logging to `log.txt`.

Phase 5 — Testing and Polish. Tested edge cases, added input validation, and improved menu prompts.

5.3 Team Responsibilities

Member	Roll No.	Responsibility
Danish Raheem	25K-6514	Data structure design, file I/O, and search functions.
Hiten Kalani	25K-6549	Menu interface, matching algorithm, logging, and testing.

6. Testing and Results

6.1 Test Cases

#	Test Case	Expected Result	Outcome
1	Add a new lost item	Item appended to lost_items.txt with auto-incremented ID.	Pass
2	Add a new found item	Item appended to found_items.txt with auto-incremented ID.	Pass
3	Search by category 'Electronics'	Only matching records are listed.	Pass
4	Run match suggestion	Lost wallet and found wallet of same category appear as a suggested pair.	Pass
5	Mark item as returned	Status updates to 'returned' and entry appears in log.txt.	Pass
6	Restart program	All previous records and statuses are reloaded from files.	Pass
7	Search for non-existent item	Program prints 'No records found' without crashing.	Pass

Table 1. Functional test cases and observed outcomes.

6.2 Results Summary

All planned features were tested with multiple sample records. The program correctly persisted data across restarts, produced sensible match suggestions, and handled empty inputs and missing records gracefully. No crashes were observed during normal use.

7. Challenges and Solutions

Reading strings with spaces. `scanf("%s", ...)` stops at whitespace, which broke fields like item names and descriptions. Resolved by switching to `fgets()` with manual newline stripping.

Choosing a delimiter. Commas appeared inside descriptions and broke CSV parsing. Switched to the pipe character (`|`) which does not appear in normal item descriptions.

Designing the matching algorithm. A pure exact-name match missed obvious pairs. The final algorithm compares category first and then performs a case-insensitive substring match on the name field, which produced more useful suggestions.

Ensuring data is saved. Early versions lost data when the user closed the terminal directly. Resolved by writing changes back to file immediately after each add or update operation, in addition to the final save on exit.

8. Deliverables

- C source code (`main.c`) implementing the full system.
- Sample data files (`lost_items.txt`, `found_items.txt`, `log.txt`) used during testing.
- This final project report.
- A live demonstration to the course instructor.

9. Relevance to Programming Fundamentals

This project applies the core constructs taught in Programming Fundamentals directly:

Structs: The `Item` struct groups eight related fields into a single record type — the foundation of the entire system.

Arrays: Two arrays of `Item` hold all lost and found records in memory during execution.

File I/O: `fopen`, `fprintf`, `fgets`, and `fclose` are used to load, save, and append records to text files.

Loops and Conditionals: `for` and `while` loops drive the menu, iterate over arrays, and parse file lines. `if` / `else` and `switch` handle menu choices and search filters.

Functions: Each operation (add, view, search, match, update, save) is implemented as its own function, keeping the code modular and readable.

Strings: `strcmp`, `strstr`, and `strcpy` from `<string.h>` drive the search and matching logic.

10. Conclusion

The Lost and Found Management System demonstrates that the constructs introduced in Programming Fundamentals — structs, arrays, file I/O, loops, conditionals, and functions — are sufficient to build a complete, useful application that solves a real institutional problem. The project gave the team practical exposure to data modelling, file handling, modular code design, and user-driven testing.

11. References

• Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*, 2nd ed., Prentice Hall.

• *cppreference.com* — C standard library reference for `<stdio.h>` and `<string.h>`.

• Course handouts and lecture notes, Programming Fundamentals, FAST NUCES Karachi, Spring 2026.